universidade de aveiro
departamento de eletrónica, telecomunicações e informática

Software Architectures

Architecture Checkpoint Report

Assignment 2: Architectural Evolution of nopCommerce

Group 107

Afonso Ferreira	113480
Tomás Brás	112665
Hugo Ribeiro	113402
Rodrigo Abreu	113626

April 30, 2026

Contents

Chapter 1

Introduction and Scenario

1.1 Scenario Choice

This project follows **Scenario C - Omnichannel Commerce Core**. VerdeMart Retail started by using nopCommerce as its online storefront, but the business now needs more than a web shop. It wants web sales, warehouse execution, shipping, store operations, support, and customer visibility to work together as part of one commerce experience.

In this scenario, nopCommerce becomes the commerce core of that wider ecosystem. A web order should be reflected in operational systems such as warehouse and shipping, and changes that happen outside nopCommerce, such as stock or fulfillment updates, should become visible again in the customer and operator experience. This is important because those surrounding systems may be slow, stale, unavailable, or inconsistent, but the commerce experience still needs to remain useful and understandable.

1.2 Core Architectural Problem

Currently, nopCommerce mainly acts as the online shop: it owns the storefront, catalog browsing, cart, checkout, customer accounts, and order creation inside one web-centered platform. This works while the business is mostly operating through the online channel, but it does not fully address an omnichannel environment where warehouse execution, shipping, store operations, and support systems also influence the customer experience.

The core architectural problem is deciding how this online shop can evolve into a commerce core without assuming that every surrounding operational system is always fast, available, and consistent. The architecture must define what remains inside nopCommerce, what moves around it, and how the system behaves when warehouse or shipping state is delayed, stale, unavailable, or later recovered.

Chapter 2

Current-State Analysis

2.1 Architecture Overview

nopCommerce follows onion architecture, where all dependencies point inward toward the core. The solution is split into four main layers: `Nop.Core`, `Nop.Data`, `Nop.Services`, and the presentation layer (`Nop.Web` and `Nop.Web.Framework`). Each layer has a specific role:

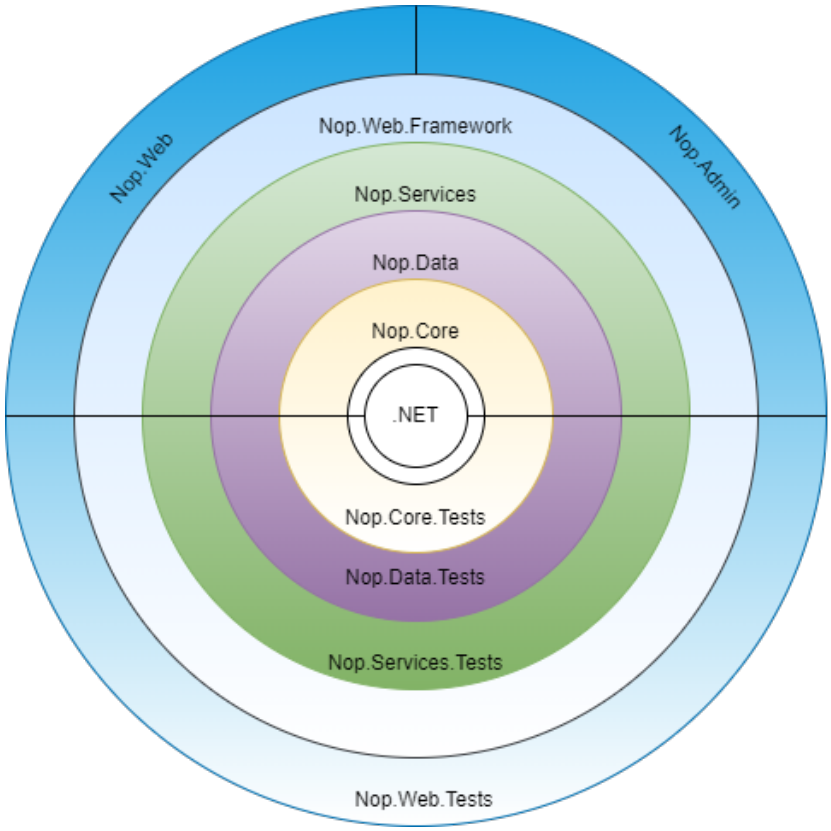

Figure 2.1: Current layered/onion architecture of nopCommerce.

- `Nop.Core`: The innermost layer. Contains domain entities, caching, events, and helper classes shared across the whole solution. It has no dependencies on any other project. Key Scenario C entities live here: `Order`, `OrderItem`, `Product`, `ProductWarehouseInventory`, `StockQuantityHistory`, `Warehouse`, and `Shipment`.
- `Nop.Data`: Separates data-access logic from business objects. nopCommerce uses a Linq2DB Code-First approach, with FluentMigrator for schema migrations and Fluent API for

persistence mappings.

- `Nop.Services`: The outermost layer of the Application Core. It holds business logic, validations, and calculations, and uses a repository pattern to expose features to layers above it. This is where order processing, inventory adjustment, shipping, and checkout logic live.
- `Nop.Web` and `Nop.Web.Framework`: The presentation layer. `Nop.Web` contains controllers, models, views, and the storefront and admin area. `Nop.Web.Framework` is a shared class library with common presentation logic used by both the public site and the admin panel.
- **Plugins**: Loaded as separate Visual Studio projects and extend nopCommerce for payments, shipping, authentication, search, and other integrations. They run inside the same runtime as the monolith.

2.2 How a Purchase Works Today

A customer order flows from the storefront controllers in `Nop.Web` down through `Nop.Services`. `OrderProcessingService` creates the order, adjusts inventory, persists state via repositories, and publishes internal events like `OrderPlacedEvent`. Shipment updates follow later through shipment services, which update order status and trigger notification events. This works cleanly when nopCommerce owns the full picture. It breaks down when an external warehouse, POS, or carrier system owns part of the order's lifecycle and may be slow, unavailable, or return conflicting data. The internal event system is also worth noting. It publishes events in-process to registered consumers and awaits each one. If a consumer fails, the error is logged and execution continues. There is no message broker and no standard durable integration retry, replay, or dead-letter mechanism. This is fine for cache invalidation or plugin reactions — it is not sufficient for reliable integration with external systems.

2.3 What Already Works for Scenario C

The current implementation already provides several Scenario C-relevant capabilities inside the monolith: web checkout and order creation, a local stock and warehouse model, shipment and tracking records, customer/admin order visibility, and plugin or scheduled-task extension points.

2.3.1 Order Capture and Customer Visibility

nopCommerce has mature order management: customer accounts, checkout, payment status, order history, shipment tracking, admin screens, and notification emails. This gives Scenario C a solid foundation. The order placement workflow is a natural seam where an integration event like "order accepted for fulfillment" can be emitted — currently it only fires in-process, but extending it with a durable outbox is straightforward without touching checkout itself.

2.3.2 Inventory and Warehouse Model

The platform already models stock quantity, reserved quantities, warehouse assignments, and stock history. This is directly useful for omnichannel stock visibility. The limitation is that nopCommerce treats this as local application state — it has no concept of whether a stock number came from the WMS ten seconds ago or ten hours ago, or whether a POS sale has already reduced it.

2.3.3 Fulfillment and Shipping

Shipment entities, tracking numbers, shipping plugins, pickup-in-store support, and customer notifications for shipping progress are all present. These support buy-online-fulfill-elsewhere flows. The gap is that shipment state is currently set by administrators or computed inside nopCommerce — there is no first-class integration status like "sent to WMS," "warehouse delayed," or "awaiting carrier confirmation."

2.3.4 Plugins and Scheduled Tasks

The architecture is modular, making it compatible with a variety of third-party services including ERPs, PIMs, and POS systems through its Web API plugin. Scheduled tasks can run background work — retries, synchronization, cleanup — and are protected against overlapping execution. These are useful extension points for Scenario C, though they lack the reliability structure needed for durable integration on their own.

2.4 What Does Not Work for Scenario C

2.4.1 No Durable Integration Path

The biggest gap is that there is no outbox, no integration message lifecycle, and no retry state. When an order is placed and the warehouse system is down, nopCommerce has no built-in guarantee that a fulfillment command was recorded, will be retried, or can be reconciled later. Logging an error is not a recovery path — operators need to know whether the order is waiting, failed, duplicated, or already accepted.

2.4.2 No Freshness or Source Metadata on External State

Stock, shipment, and fulfillment data can come from WMS, POS, ERP, or manual admin updates. The current model stores a value but does not record where it came from, when it was last confirmed, or whether it conflicts with another source. Stale data looks identical to fresh data to both customers and operators.

2.4.3 Synchronous Plugin Calls Expose the Customer to External Failures

When a plugin calls an external system during checkout or shipping-rate calculation, a slow or failed response degrades the customer experience directly. Scenario C requires the commerce core to stay useful when surrounding systems degrade — accepting safe work, signaling degraded status, and deferring unsafe steps — rather than failing visibly.

2.4.4 No Standard Failure State or Reconciliation Model

There is no common status model for integration messages: no "pending," "retrying," "failed," or "dead-lettered" states. There is no correlation between a placed order, the fulfillment command sent from it, and the acknowledgement (or conflict) that comes back. This makes recovery from partial failures opaque.

2.4.5 Shared Database Must Stay Internal

The nopCommerce database blends catalog, orders, customers, stock, shipments, configuration, and logs. This is fine for a monolith. It becomes a problem if external adapters start reading or writing it directly. Presentation logic in `Nop.Web` is isolated from the database — controllers only reference services, not repositories directly. The same discipline must apply at system boundaries:

any fulfillment adapter, stock projection, or integration worker must communicate through APIs or events, not by coupling to nopCommerce tables.

2.5 Useful Seams for Evolution

The architecture provides clear attachment points without requiring a rewrite:

- **Order placement**: emit a durable integration message for fulfillment and ERP immediately after an order is accepted.
- **Shipment updates**: receive warehouse and carrier progress and update nopCommerce shipment projections and customer-visible status.
- **Inventory**: receive stock changes from WMS or POS and update a customer-facing availability view with freshness metadata.
- **Scheduled tasks**: run outbox publishing, retry logic, timeout detection, and reconciliation outside the customer request path.
- **Plugins and APIs**: connect to surrounding systems through explicit adapters, not direct database access.
- **Admin visibility**: expose integration status, failures, and reconciliation actions to operators.

2.6 What This Means for the Target Architecture

nopCommerce should remain the owner of checkout, order management, customer visibility, catalog, and administration. Moving those out would create unnecessary scope. The evolution targets three specific gaps instead:

- **Reliable propagation**: fulfillment and ERP commands must leave through a durable asynchronous path, not in-process events or direct plugin calls.
- **Visible degradation**: when WMS, POS, ERP, or carriers are slow, unavailable, or contradictory, the system should expose an honest state — pending, delayed, degraded, or reconciliation required — rather than silently serving stale data.
- **Recovery and reconciliation**: retries, idempotency, correlation identifiers, and operator-visible failure records should make it possible to recover without duplicating orders or hiding conflicts.

The current state is a strong starting point: it provides a real commerce base with genuine seams. The evolution adds reliability and cross-channel state visibility around the existing monolith rather than replacing it.

Chapter 3

Domain and Bounded Contexts

3.1 Domain Overview

3.2 Bounded Contexts

3.3 Responsibilities

3.4 Architecture Diagram

Chapter 4

Quality Attribute Scenarios

Quality attribute scenarios make the architectural drivers measurable. For the VerdeMart omnichannel scenario, the most important pressure is not only that nopCommerce must create orders correctly, but that the commerce experience must remain understandable while warehouse, shipping, store, and support systems are slow, stale, unavailable, or later recovering.

The scenarios below use the six-part structure from the course material: stimulus source, stimulus, artifact, environment, response, and response measure.

4.1 Architectural Drivers

The scenarios are driven by four architectural concerns:

- **Availability:** web orders must not be lost when warehouse or shipping systems fail.
- **Consistency:** inventory and order state may be temporarily stale, but inconsistencies must be visible and recoverable.
- **Performance:** customer-facing operations must stay responsive during backend synchronization and sales peaks.
- **Modifiability:** new operational channels must be integrated without rewriting checkout or sharing the nopCommerce database.

4.2 Scenario 1 - Availability under Warehouse Failure

Driver: Availability and operational continuity.

Part	VerdeMart scenario
Stimulus source	Warehouse execution system
Stimulus	The warehouse API takes more than 10 seconds to respond or rejects fulfillment requests for newly paid web orders.
Artifact	Fulfillment synchronization and the order status view shown to customers and operators
Environment	Friday afternoon peak traffic, with 3 times normal order volume, while the warehouse dependency is degraded but the web shop is still accepting orders
Response	The order is accepted and persisted locally; the fulfillment request is stored for asynchronous retry; the order is shown as awaiting warehouse confirmation; subsequent orders are not blocked by the degraded dependency.
Response measure	No paid order is lost; no duplicate fulfillment request is sent; pending fulfillment state is visible to support within 30 seconds; retry or escalation starts within 5 minutes.

4.3 Scenario 2 - Consistency under Stale Inventory

Driver: Consistency and recoverable inventory state.

Part	VerdeMart scenario
Stimulus source	Store point-of-sale system and warehouse stock service
Stimulus	A store sale reduces stock by 5 units, but the update reaches nopCommerce 3 minutes after the web catalog still showed those units as available.
Artifact	Inventory availability view used by product pages, cart validation, and checkout
Environment	Concurrent web and physical-store sales for the same product during a promotion, with external stock updates delayed by up to 3 minutes
Response	The commerce core marks stock data as stale, revalidates stock before checkout completion, preserves already-created orders for reconciliation, and records an inventory discrepancy for recovery.
Response measure	Product pages show stale stock status within 30 seconds of detection; overselling stays under 1% of affected order lines; inventory discrepancies are queued for reconciliation within 1 minute.

4.4 Scenario 3 - Performance during Omnichannel Order Bursts

Driver: Performance under customer-facing load.

Part	VerdeMart scenario
Stimulus source	Web customers, store operators, and customer support agents
Stimulus	A campaign creates 5 times normal traffic for product browsing, cart validation, order creation, and order-status queries.
Artifact	Storefront, checkout path, order status view, and integration event publishing path
Environment	Peak campaign period, with 5 times normal traffic and background synchronization still running
Response	The system prioritizes checkout writes, serves order-status reads from a projection, and processes warehouse and shipping synchronization asynchronously without blocking the customer request path.
Response measure	Checkout p95 latency remains below 2 seconds; order-status read p95 remains below 1 second; at least 99% of integration events are published or queued within 60 seconds.

4.5 Scenario 4 - Recoverability after Shipping Outage

Driver: Recoverability after external dependency failure.

Part	VerdeMart scenario
Stimulus source	Shipping provider or shipping adapter
Stimulus	Shipment label creation fails for 50 ready-to-dispatch orders and later becomes available again.
Artifact	Shipping integration workflow, retry queue, and order shipment status
Environment	Warehouse has packed orders, but the external shipping provider is unavailable for 20 minutes
Response	The system stores failed label requests, keeps orders in a clear awaiting-shipping state, avoids repeated synchronous calls from operators, and resumes processing automatically when the provider recovers.
Response measure	100% of failed label requests are retained; operators see the outage state within 30 seconds; 95% of queued labels are created within 10 minutes after provider recovery.

4.6 Scenario 5 - Modifiability for New Operational Channels

Driver: Modifiability and controlled integration growth.

Part	VerdeMart scenario
Stimulus source	VerdeMart business team and integration developers
Stimulus	A new store-operations system must receive order-ready-for-pickup events and send pickup-completed updates back to the commerce core.
Artifact	Integration layer, domain events, and order status update boundary
Environment	Existing web, warehouse, and shipping integrations are already in production
Response	The architecture allows the new channel to be added through a new adapter and event subscription without changing checkout logic or sharing the nopCommerce database with the external system.
Response measure	The new integration requires changes in no more than one adapter and one application-service boundary; no direct external database access is introduced; regression tests for checkout and existing integrations remain green.

4.7 Traceability to Architectural Drivers

Scenario	Architectural driver
Availability under warehouse failure	Availability, recoverability
Consistency under stale inventory	Consistency
Performance during omnichannel order bursts	Performance
Recoverability after shipping outage	Recoverability, availability
Modifiability for new operational channels	Modifiability

Chapter 5

Architectural Approach

5.1 Chosen Framework

The selected architectural design framework is **Attribute-Driven Design (ADD)**.

ADD fits this project because the VerdeMart scenario is mainly driven by measurable quality attribute pressures. The main question is not only which components should exist, but how the commerce core should behave when surrounding operational systems are delayed, stale, unavailable, or recovering.

In this project, ADD is applied as follows:

1. Drivers identified: availability, consistency, performance, recoverability, and modifiability.
2. Five quality attribute scenarios defined, each with a measurable response.
3. Tactics selected per scenario: asynchronous integration, transactional outbox-style event publishing, read-model separation, retry queue, reconciliation, adapter pattern, and observability.
4. Target architecture decomposed around those tactics: nopCommerce Core, Integration Layer, Operational Adapters, External Systems, and Support and Operations View.
5. Validation focused on the riskiest path: asynchronous propagation, degraded dependency handling, retry, and recovery.

5.2 Justification

The chosen scenario requires nopCommerce to evolve from a web storefront into the commerce core of a wider operational ecosystem. This creates architectural pressure in five main areas: availability, consistency, performance, recoverability, and modifiability.

ADD is appropriate because each of these pressures can be expressed as a concrete quality attribute scenario. Warehouse failure drives the need for asynchronous fulfillment and retry. Stale inventory drives the need for inventory visibility, revalidation, and reconciliation. Order bursts drive the need to protect customer-facing paths from slow backend synchronization. Shipping outage drives the need for durable retry and explicit recovery state. New operational channels drive the need for adapters and stable integration boundaries.

This gives a direct trace from the problem to the architecture:

ADD input	Scenario	Architectural consequence
Availability	Scenario 1	Avoid synchronous dependency on warehouse systems during checkout.
Consistency	Scenario 2	Introduce explicit inventory state, stale-state visibility, and reconciliation.
Performance	Scenario 3	Keep checkout and order reads separate from slow integration processing.
Recoverability	Scenario 4	Store failed integration work durably and retry after dependency recovery.
Modifiability	Scenario 5	Add external systems through adapters instead of direct database coupling.

5.3 Why Not ACDM or ADM as the Main Framework

ACDM was considered because it is useful for architectural decision-making, risk discussion, and stakeholder validation. However, it is not the main framework for this project because the strongest design input is the set of measurable quality attribute scenarios. The project must show runtime behavior under failure, delay, stale state, and recovery; ADD gives a more direct path from those scenarios to concrete architectural tactics and component responsibilities.

ACDM-style reasoning is still useful later in the project, especially when documenting ADRs and explaining rejected alternatives. For example, the decision to use asynchronous integration can be recorded as an ADR with alternatives such as direct synchronous calls or shared database access. However, the decision itself is primarily driven by the availability and recoverability scenarios, which makes ADD the better primary method.

ADM/TOGAF was also considered, but it is broader than necessary for this assignment. ADM is better suited to enterprise-wide transformation across business, data, application, and technology architecture. VerdeMart does have an omnichannel business context, but the scope of this work is a focused software architecture evolution of nopCommerce, not a full enterprise architecture programme.

5.4 How ADD Shapes the Target Architecture

Using ADD means that each architectural element in the target design should answer a specific quality attribute pressure. The surrounding systems are not added only because they exist in the business scenario; they are added because they create design pressure that nopCommerce must handle explicitly.

The target architecture emphasizes asynchronous integration because Scenario 1 requires that a warehouse timeout does not block checkout. A direct synchronous call from checkout to the warehouse cannot satisfy that constraint, so accepted orders are propagated through a durable integration path.

It also makes status explicit for pending, degraded, failed, and recovered work because Scenarios 1 and 4 require support agents and operators to understand what is happening during warehouse or shipping failure. Retry and reconciliation are part of the architecture because Scenarios 2 and 4 assume stale inventory, failed label creation, and later recovery. External systems are connected through adapters because Scenario 5 requires new operational channels to be added without rewriting checkout or sharing the nopCommerce database.

The next chapter uses these ADD-driven conclusions to define the target architecture.

Chapter 6

Target Architecture

6.1 Overview

The target architecture keeps nopCommerce as the commerce core, but stops treating every surrounding operational capability as a direct synchronous dependency. nopCommerce remains responsible for the customer-facing commerce experience: catalog browsing, cart, checkout, customer accounts, payment result handling, and order creation. Around it, the architecture adds integration boundaries for warehouse, inventory, shipping, store operations, and support visibility.

The main architectural change is the introduction of an integration layer between nopCommerce and the surrounding systems. This layer exposes adapters for external systems, publishes and consumes integration events, stores work that cannot be completed immediately, and retries or reconciles failed operations. This allows the commerce core to remain useful when warehouse, shipping, or store systems are slow, stale, unavailable, or recovering.

The architecture is therefore not a full microservice rewrite of nopCommerce. It is an evolutionary architecture where the monolith keeps the responsibilities that are already stable inside nopCommerce, while volatile omnichannel coordination is moved to explicit integration boundaries.

6.2 Main Components and Services

nopCommerce Commerce Core is the main application and remains the system of record for web customers, shopping carts, checkout, payments, and created orders. It owns the customer-facing transaction and decides whether an order can be accepted.

Integration Layer is responsible for communication between nopCommerce and external operational systems. It receives events from the commerce core, routes them to the correct adapter, stores pending integration work, and records failures for retry or reconciliation.

Warehouse Adapter translates accepted orders into fulfillment requests for the warehouse execution system. It also receives fulfillment updates such as picked, packed, or failed fulfillment.

Inventory Adapter synchronizes stock changes from warehouse and store systems into an inventory availability view used by nopCommerce. It is responsible for detecting stale or conflicting stock information and triggering reconciliation.

Shipping Adapter manages shipment label creation and shipping status updates. When the shipping provider is unavailable, it stores failed label requests and retries them later.

Store Operations Adapter represents physical-store capabilities such as pickup, in-store stock changes, and pickup-completed updates. It allows store operations to participate in the commerce flow without accessing the nopCommerce database directly.

Support and Operations View gives support agents and operators visibility over pending, degraded, failed, and recovered work. This is important because a degraded omnichannel system

must be understandable, not only technically resilient.

6.3 Synchronous and Asynchronous Interactions

The architecture uses synchronous interactions only when the user needs an immediate answer and the dependency is inside the commerce core. Product browsing, cart operations, checkout validation, payment result handling, and order creation remain synchronous from the customer's point of view. These operations must be fast and predictable because they directly affect the buying experience.

Interactions with warehouse, shipping, inventory synchronization, and store operations are primarily asynchronous. After an order is created, nopCommerce publishes an order-created event or stores an equivalent integration task. The integration layer then sends the fulfillment request to the warehouse adapter. If the warehouse is slow or unavailable, the order is not lost and the checkout path is not blocked; the fulfillment state becomes pending or degraded and is retried later.

Inventory updates also follow an asynchronous model. Warehouse and store systems publish stock changes, and the inventory adapter updates the availability view used by nopCommerce. Because this data may be stale, checkout still performs stock validation before completion, and detected conflicts are sent to reconciliation.

Shipping follows the same pattern. When the warehouse marks an order as ready to dispatch, the shipping adapter requests the label. If the provider fails, the label request is stored, the order is marked as awaiting shipping, and automatic retry resumes when the provider recovers.

6.4 Data Ownership

Data ownership is explicit to avoid hidden coupling between systems. nopCommerce owns customer accounts, carts, checkout state, payment result state, and the canonical web order. External systems are not allowed to read or write the nopCommerce database directly.

The warehouse system owns physical fulfillment state, such as picking, packing, dispatch preparation, and fulfillment failures. The shipping provider owns carrier label generation and carrier tracking updates. Store operations own in-store events such as pickup handling and point-of-sale stock changes. The integration layer owns integration metadata: pending work, retry attempts, correlation identifiers, failed messages, and reconciliation records.

nopCommerce may keep local views of external state, such as order fulfillment status, inventory availability, or shipping status, but these are projections used to support customer and operator visibility. They are not shared databases and should be updated through integration messages or adapter APIs.

6.5 Cross-Cutting Concerns

Reliability shapes the design because warehouse, shipping, and store systems can fail independently. Integration work must be durable, retries must be controlled, and repeated messages must not create duplicate fulfillment or shipping actions.

Observability is required because degraded state must be visible. Operators and support agents need to know whether an order is waiting for warehouse confirmation, shipping label creation, inventory reconciliation, or manual intervention.

Consistency is handled through explicit stale-state handling and reconciliation rather than by assuming immediate consistency across all channels. This is necessary because web, warehouse, and store operations can observe stock at different times.

Security and access control remain important because surrounding systems should not get direct database access. They interact through adapters, APIs, or events with controlled permissions.

Modifiability is supported by keeping external systems behind adapters. A new POS, warehouse, or shipping provider can be added by implementing or replacing an adapter without rewriting checkout or changing the core nopCommerce data model.

6.6 Main Architecture Diagram

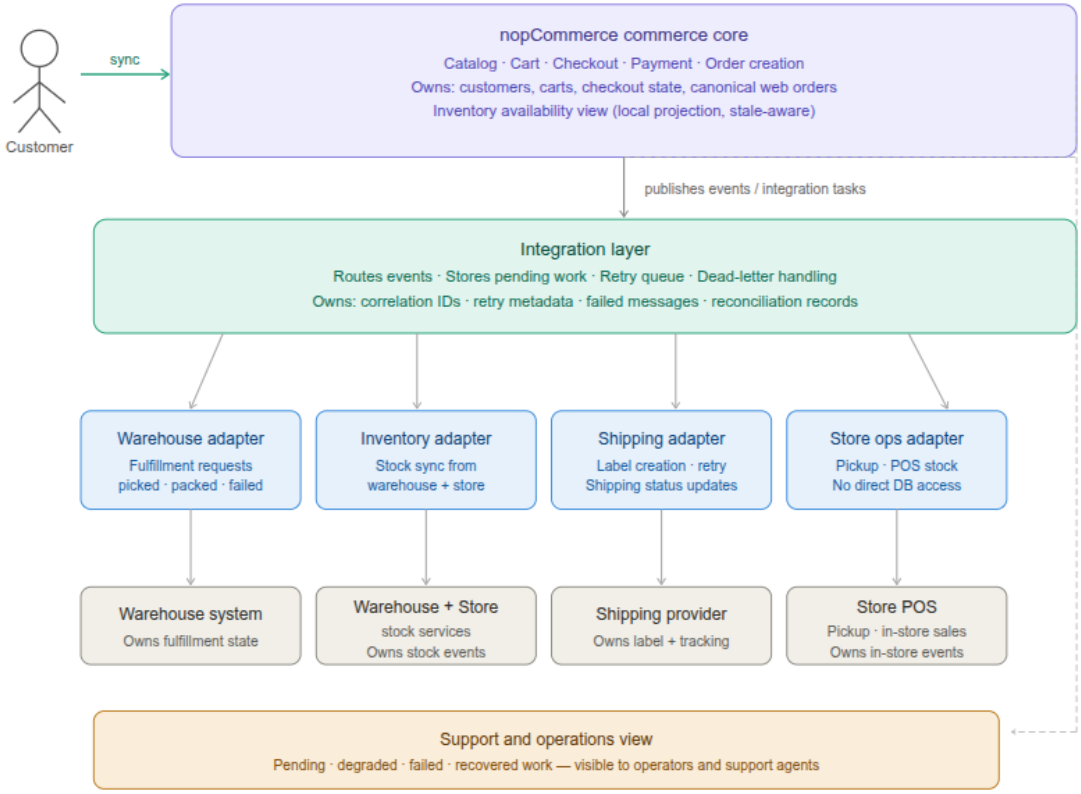

Figure 6.1: Target architecture for Scenario C: nopCommerce as an omnichannel commerce core.

The architecture diagram highlights the main design rule: customer-facing commerce remains inside nopCommerce, while operational coordination with warehouse, shipping, inventory, and store systems happens through explicit integration boundaries.

It also distinguishes customer-facing synchronous flows from asynchronous integration flows. This separation is important because checkout and order visibility must remain responsive even when external systems are slow or unavailable. Data ownership is explicit: nopCommerce owns customers, carts, checkout, payments, and canonical web orders; the integration layer owns retry metadata, correlation identifiers, dead letters, and reconciliation records; external systems own their operational state.

Chapter 7

Architectural Decisions

7.1 ADR 1 - XXX

7.1.1 Context

7.1.2 Decision

7.1.3 Alternatives Considered

7.1.4 Consequences

7.2 ADR 2 - XXX

7.2.1 Context

7.2.2 Decision

7.2.3 Alternatives Considered

7.2.4 Consequences

7.3 ADR 3 - XXX

7.3.1 Context

7.3.2 Decision

7.3.3 Alternatives Considered

7.3.4 Consequences

Chapter 8

Risk and Validation Plan

8.1 Purpose

This chapter identifies the main architectural risks in evolving the current nopCommerce implementation into the Scenario C omnichannel commerce core described in the target architecture. The current implementation is treated as the baseline: nopCommerce already supports the customer-facing commerce flow, but the planned integration layer, adapters, retry metadata, reconciliation records, and operational visibility still need to be validated before they can be claimed as implemented architecture.

The validation plan therefore focuses on the pressure required by the assignment: surrounding operational systems may be delayed, stale, unavailable, or contradictory, while the commerce core must remain useful and understandable.

8.2 Main Architectural Risks

Risk	**Why it matters**	**Related scenarios**
Surrounding systems are unavailable or delayed while checkout remains active	Warehouse or shipping failures must not cause paid orders to be lost or block later customers.	Availability under warehouse failure; recoverability after shipping outage
Inventory is stale or contradictory across online, store, and warehouse channels	The customer experience can become misleading if stock shown online no longer matches physical stock.	Consistency under stale inventory
Retry, replay, or idempotency is insufficient for asynchronous integration work	Failed integration work can be lost, repeated, or duplicated if durable retry and correlation are not designed carefully.	Availability; recoverability
Pending, degraded, failed, or recovered states are not visible to operators	A resilient system is still hard to operate if support teams cannot explain order or stock state to customers.	Availability; recoverability; observability
Integration work slows customer-facing operations	Omnichannel synchronization must not make checkout and order-status views depend on slow external systems.	Performance during omnichannel order bursts
Future adapters bypass boundaries or share the nopCommerce database	Direct coupling would make new channels harder to add and would weaken data ownership between systems.	Modifiability for new operational channels

8.3 Validation Activities

Validation activity	What it proves	Evidence to collect
Simulate a warehouse or shipping timeout during order processing	A paid order is persisted locally, marked as pending or degraded, and not lost when the external dependency fails.	Order screenshot, logs, retry record, timestamps
Simulate a delayed stock update from a store or warehouse system	The system can detect stale stock, expose the state, and create a reconciliation path instead of assuming immediate consistency.	Before and after stock views, discrepancy or reconciliation record
Verify retry, replay, and idempotency behavior for failed integration work	Repeated attempts do not create duplicate fulfillment or shipping actions, and failed work can be retried after recovery.	Retry attempts, correlation identifier, duplicate-prevention result
Verify support and operations visibility	Operators can distinguish normal, pending, degraded, failed, and recovered work without inspecting internal database tables.	Support or admin view screenshots, status transitions
Run a small order-burst check while background synchronization is active	Customer-facing checkout and order-status reads remain responsive while asynchronous work is queued or processed.	Latency measurements, queued work count, test notes
Review integration boundaries	Surrounding systems use adapters, APIs, or events and do not read or write the nopCommerce database directly.	Architecture review notes, code or configuration references

8.4 Feasibility Spike Focus

The feasibility spike should target the riskiest part of the target architecture: durable asynchronous order propagation from nopCommerce to a surrounding operational capability when that capability is slow or unavailable. The spike can use a simulator or stub for the warehouse or shipping dependency if a real system is not available.

The spike should start from a placed order or an equivalent order-created event. The external adapter should then be made unavailable or slow. The expected result is that the commerce core keeps the order, records pending integration work, exposes a degraded or awaiting-confirmation state, and later retries successfully when the dependency recovers. Chapter 10 should contain the concrete implementation details and evidence for this spike.

Chapter 9

Evolution Roadmap

Chapter 10

Feasibility Spike

10.1 What Was Tested

10.2 What Worked

10.3 Evidence